HealthTracker Project Logic Documentation

1) Project Scope

HealthTracker is an offline-first personal health app built with .NET MAUI. The project focuses on medicine management, dose tracking, health metrics, doctor visits, report storage/scanning, timeline/history views, AI-assisted extraction/chat, backup/export, and app-level PIN lock.

Primary solution files:

- `HealthTracker/HealthTracker.sln`
- `HealthTracker/HealthTracker.csproj` (main app, Android target)
- `HealthTracker.Tests/HealthTracker.Tests.csproj` (unit tests)

2) Technical Architecture

Architecture style:

- MVVM: `Views` + `ViewModels`
- Service layer for business logic
- Repository layer for persistence
- SQLite local storage via `sqlite-net-pcl`
- Dependency Injection configured in `MauiProgram.cs`

Dependency flow:

- View -> ViewModel -> Service -> Repository -> DatabaseService -> SQLite

Core registration location:

- `HealthTracker/MauiProgram.cs`

Key external libraries:

- `Plugin.LocalNotification` for reminders
- `Plugin.Maui.OCR` for OCR
- `QuestPDF` for health report PDF generation
- `Microcharts.Maui` for metrics charting
- `UglyToad.PdfPig` for PDF text extraction

3) Startup and App Lifecycle Logic

Defined in `HealthTracker/App.xaml.cs`:

- App starts with `AppShell`.
- On start:
- Requests notification permission.
- Checks Android exact alarm permission (Android 12+), optionally opens settings.
- Reschedules all reminders.
- Enforces PIN lock if PIN exists.
- On sleep: stores background timestamp.
- On resume: if backgrounded >= `2` minutes (`BackgroundResumeThresholdMinutes`), enforces PIN lock again.

Navigation shell and primary tabs are defined in `HealthTracker/AppShell.xaml`:

- Today
- Metrics
- Reports
- Doctors
- Timeline

Toolbar actions:

- Ask AI
- Settings

4) Persistence and Data Model Logic

Database file:

- `healthapp.db` in app data directory (`FileStorageHelper.GetDatabasePath()`).

Database initialization and migration:

- `HealthTracker/Services/DatabaseService.cs`
- Creates tables for medicines, schedules, logs, courses, metrics, reports, report pages, doctor visits, pending medicines.
- Contains migration logic for legacy columns:
- Removes `DoseAmount` from `MedicineSchedule` if present.
- Removes `Dosage` from `Medicine` and ensures `IsOngoing` exists.

Core model entities:

- `Medicine`: medicine definition, stock, duration/course settings, active flag, sort order.
- `MedicineSchedule`: daily time slots per medicine.
- `MedicineLog`: taken/missed records and stock before/after snapshot.
- `MedicineCourse`: a treatment period with planned vs taken doses.
- `HealthMetric`: metric type/value(s)/timestamp/notes.
- `DoctorVisit`: visit metadata + optional prescription image path.
- `MedicalReport`: report metadata + primary stored file path.
- `ReportPage`: multi-page report file rows.
- `PendingMedicine`: OCR-extracted medicine candidates awaiting approval.
- `TimelineEntry`: aggregated UI event item.

Repository pattern:

- Base CRUD in `Repositories/BaseRepository.cs`.
- Per-entity repositories provide query logic (active/inactive medicine, ordered lists, range logs, etc.).

5) Business Rules and Validation Logic

Key constants in `Constants/AppConstants.cs` and `Constants/MetricTypes.cs`.

Metric validation (`Helpers/ValidationHelper.cs`):

- Blood pressure uses systolic + diastolic required ranges.
- Other metrics use per-type min/max ranges.

File type validation (`ValidationHelper.IsValidFileType`):

- Allowed: `jpg`, `jpeg`, `png`, `pdf`.

PIN validation (`ValidationHelper.IsValidPin`):

- Length: 4 to 6 digits.
- Digits only.

File storage layout (`Helpers/FileStorageHelper.cs`):

- Base folder: `HealthApp` under app data.
- Subfolders by report type: reports, prescriptions, scans, other.
- File naming includes type/date/random suffix.

6) Core Service Logic

6.1 MedicineService (`Services/MedicineService.cs`)

Responsibilities:

- Add/edit/delete/deactivate medicines.
- Manage schedules and treatment courses.
- Dose logging and stock reduction.
- Auto-miss logging and course completion checks.
- Adherence stats and backfilling past doses.

Important behaviors:

- Adding medicine requires at least one schedule.
- If non-ongoing and end date not provided, computes end date from prescribed days.
- Creates initial active course on medicine creation.
- Schedules notifications immediately after add/update/start-new-course.
- `CheckAndCompleteCourses()` completes course and deactivates medicine when end criteria met.
- `LogDoseTaken()` decrements stock by 1, inserts taken log, increments active course `TotalTaken`.
- `AutoLogMissedDoses()` creates missed logs for yesterday for unlogged schedules and extends end date proportionally.
- `BackfillPastDoses()` inserts taken logs historically up to now (skips future schedule slots) and updates stock/course totals.

6.2 ReminderService (`Services/ReminderService.cs`)

Responsibilities:

- Schedule and cancel local notifications.

Behavior:

- Each schedule creates two daily reminders:
- Early reminder at -15 minutes.
- On-time reminder.
- Notification IDs are deterministic from medicine/schedule IDs.
- `CancelDoseReminders()` cancels current reminders for a schedule and immediately reschedules from next day.
- `RescheduleAll()` cancels all notifications then re-schedules active, non-expired medicines.

6.3 MetricsService (`Services/MetricsService.cs`)

Responsibilities:

- Add/read/update/delete health metrics.

Behavior:

- Enforces metric validation before add/update.
- Exposes latest-by-type, history, and latest snapshot for all metric types.

6.4 ReportService (`Services/ReportService.cs`)

Responsibilities:

- Persist report metadata and copy source files into app storage.
- Support single-page and multi-page reports.

Behavior:

- Validates pages and file types.
- Copies first page as `MedicalReport.FilePath` primary file.
- Inserts each page into `ReportPage` table.
- Delete removes all page files and DB rows.

6.5 DoctorService (`Services/DoctorService.cs`)

Responsibilities:

- Create/update/delete/get doctor visits.

Behavior:

- Enforces follow-up date not in past.
- Optional prescription image is copied into app storage on save.
- Delete also removes saved prescription image file if present.

6.6 TimelineService (`Services/TimelineService.cs`)

Responsibilities:

- Build unified event feed.

Sources aggregated:

- Doctor visits
- Reports
- Metrics
- Dose logs
- Medicine added
- Course completed

Behavior:

- Creates `TimelineEntry` items with tag/icon/color metadata.
- Uses date grouping label `MonthYearGroup`.

- Sorts descending by `SortKey` and supports paging (`pageSize`, `offset`).

6.7 OCR/Parser Services

Prescription parser (`Services/PrescriptionParserService.cs`):

- OCR text extraction via `IOcrService`.
- Regex heuristics detect medicine lines, dosage, frequency, and form hints.
- Outputs list of `ParsedMedicine`.

Report text parser (`Services/ReportTextParserService.cs`):

- Extracts doctor, hospital, date, report type, title from OCR text via regex rules.

PDF text extractor (`Services/PdfTextExtractorService.cs`):

- Reads text page-by-page from PDF files using PdfPig.

6.8 AI Services

AI parser (`Services/AiParserService.cs`):

- Stores Gemini API key in app `Preferences` (`gemini_api_key`).
- Calls Gemini model endpoint for report/doctor-visit JSON extraction.
- Handles JSON fence cleanup and parse errors.

AI chat (`Services/AiChatService.cs`):

- Builds large contextual prompt from local DB data:
- Active/past medicines
- 3-day medicine log summary
- Doctor visits
- Recent reports (with PDF text extraction)
- Recent health metrics
- Adds conversation history and user question.
- Calls Gemini and returns response text.

6.9 Backup and Export

Backup service (`Services/BackupService.cs`):

- Creates temp folder in cache.
- Copies database + files directory.
- Zips and triggers system share sheet.

Health report service (`Services/HealthReportService.cs`):

- Generates styled PDF summary with QuestPDF.
- Sections include medicines, logs, visits, metrics, reports.

- Optional embedding of report files/images.
- On Android, can render PDF pages to images before embedding.
- Warns when generated report exceeds 50MB.

7) ViewModel Logic (UI Behavior)

7.1 Today Flow

`TodayViewModel`:

- Loads active medicine schedules for today.
- Runs `AutoLogMissedDoses()` and `CheckAndCompleteCourses()` before rendering.
- Skips schedule slots already passed at creation time for medicines created today.
- Sorts schedule cards by urgency/window.
- `TakeDose` checks stock, logs taken dose, cancels/reschedules that schedule reminder.

7.2 Medicine Flow

`MedicineListViewModel`:

- Loads active medicines with progress, adherence, stock days.
- Loads inactive medicines as past list.
- Loads pending OCR medicines for approval.
- Supports drag-drop reorder and persists `SortOrder`.

`AddMedicineViewModel`:

- Validates name, prescribed days, stock, schedule presence.
- Saves medicine + schedules + initial course.
- If start date is in past, offers backfill of past doses.

`MedicineDetailViewModel`:

- Shows schedules and today log status.
- Shows recent logs and course history summary.
- Allows start new course.
- Allows manual backfill from start date to yesterday.

`EditMedicineViewModel`:

- Loads medicine + schedules into editable fields.
- Updates medicine/schedules and reminder schedule.

7.3 Metrics Flow

`MetricsDashboardViewModel`:

- Loads latest/previous values for each metric type.
- Calculates trend text and tile styling.

`AddMetricViewModel`:

- Creates metric record with per-type validation.

`MetricHistoryViewModel`:

- Loads up to 30 points, computes latest/min/max text, builds line chart.
- Supports metric deletion.

`EditMetricViewModel`:

- Updates selected metric with validation.

7.4 Report Flow

`ReportsViewModel`:

- Loads reports list and supports deletion.

`AddReportViewModel`:

- Adds pages from file picker or camera.
- OCR runs for image pages (not PDFs).
- If AI key exists, attempts AI extraction first; falls back to regex parser.
- Auto-fills title/doctor/hospital/type/date when available.
- Saves multi-page report and metadata.

7.5 Doctor Visit Flow

`DoctorVisitsViewModel` and `DoctorVisitDetailViewModel`:

- List + detail display with delete/navigation.

`AddDoctorVisitViewModel`:

- Optional image pick/capture.
- OCR extraction from image.
- AI parsing if configured; fallback regex extraction.
- Saves visit with follow-up/date validations.

`EditDoctorVisitViewModel`:

- Loads and updates existing visit fields.

7.6 Prescription Scan to Pending

`ScanPrescriptionViewModel`:

- Scans image from gallery/camera.
- OCR + parse medicines.
- User selects entries and saves to `PendingMedicine` for later review.

7.7 History and Timeline

`HistoryViewModel`:

- Category-based history: Medicine, Doctor, Reports.
- Date-range filtering.
- Medicine history includes course overlap logic and progress summaries.

`TimelineViewModel`:

- Paged timeline loading using `TimelineService`.
- Prevents duplicate month-group labels across page boundaries.

7.8 Security and Settings

`PinLockViewModel`:

- Compares entered PIN with secure storage.
- Tracks failed attempts and enforces cooldown after max attempts.

`SettingsViewModel`:

- Save/remove PIN.
- Save/remove Gemini API key.
- Create backup ZIP.
- Generate PDF health report with/without embedded files.
- Delete all app data with double confirmation.

`AskAiViewModel`:

- Chat UI state and history management.
- Uses `AiChatService` only when API key configured.

8) Android Widget Logic

Files:

- `Platforms/Android/HealthWidget.cs`
- `Platforms/Android/WidgetDataHelper.cs`

Behavior:

- Widget shows:
- Today date

- Dose completion progress
- Next dose text
- Latest heart rate
- Latest blood pressure
- Tap opens app.
- Data is read directly from SQLite in read-only mode.

9) Routing and Navigation Logic

Primary route registration in `AppShell.xaml.cs` for detail/add/edit pages.

Main navigation patterns:

- Tab navigation for core modules.
- Push navigation for settings, AI chat, detail/edit forms.

10) Test Coverage (Current)

Unit tests exist for:

- Helpers:
- DateTimeHelper
- ValidationHelper
- ViewModels/models:
- BaseViewModel
- MedicineItem
- TodayScheduleItem
- Services:
- MedicineService

Test project:

- `HealthTracker.Tests`
- xUnit + NSubstitute + sqlite-net package references

11) Data Safety and Operational Notes

- App is offline-first and single-user.
- Health data is local SQLite + app-local files.
- No first-party cloud sync logic exists.
- AI features require user-provided Gemini API key and network access to Google endpoint.
- Backup/export and report sharing rely on platform share sheet.

12) End-to-End Logic Summary

Primary critical flows:

- Medicine lifecycle: add -> schedule reminders -> daily logging -> stock/course updates -> auto-complete/deactivate.
- Daily execution: auto-log missed doses + course completion checks + today scheduling/prioritization.
- Metrics lifecycle: validated insert/update -> dashboard trend + history chart.
- Report lifecycle: file capture/pick -> OCR/AI extraction -> stored pages + metadata -> optional embedding in generated health PDF.
- Doctor lifecycle: visit records + optional scanned prescription.
- Security lifecycle: optional PIN lock at startup and long resume gaps.
- Insight lifecycle: timeline/history aggregation + AI Q&A over local data context.

Generated on 2026-03-09 12:39